.NET Developer — 6+ Years Experience

Interview Question Bank and Answer Notes

How to Use This Handbook

This handbook is designed for taking a practical interview.

The questions should be easy for the candidate to understand. The difficulty should come from the **technical problem and depth of knowledge**, not from complicated wording.

Important interviewing rule

A candidate may understand a concept even if they do not remember the exact technical term.

For example, instead of asking:

> What is the Liskov Substitution Principle?

You can ask:

> Suppose a method accepts a `Payment` object. If I pass a `CreditCardPayment`, `UPIPayment`, or another derived type, what should we expect from those classes?

If the candidate explains that the derived classes should work correctly without breaking the expected behavior, they understand the concept.

Scoring

Score each answer from **0 to 3**.

Score	Meaning
0	Cannot explain or gives an incorrect answer
1	Knows the basic definition but lacks practical understanding
2	Good understanding with practical explanation
3	Strong understanding, including trade-offs, problems, and real-world examples

Do not ask all questions. Use this as a question bank.

---

1. OOP

Basic

1. What are the main concepts of Object-Oriented Programming?

Expected answer:

The main concepts are:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

A good candidate should be able to explain each using a simple example.

Example

```csharp id="r2jv2m"
public abstract class Payment
{
    public abstract Task ProcessAsync();
}

public class CreditCardPayment : Payment
{
    public override Task ProcessAsync()
    {
        // Process card payment
        return Task.CompletedTask;
    }
}
```

```
`Payment` provides an abstraction. Different payment types can inherit from it
and implement their own behavior.

---

### 2. What is encapsulation? Can you give a practical example?

**Expected answer:**

Encapsulation means keeping an object's internal state protected and controlling
how it can be changed.

For example, instead of allowing anyone to directly change an account balance:

```csharp id="qng2b0"
public decimal Balance { get; set; }
```

We can control it:

```csharp id="5w5b5p"
public class BankAccount
{
    private decimal _balance;
```
```

```
public decimal Balance => _balance;

public void Deposit(decimal amount)
{
    if (amount <= 0)
        throw new ArgumentException();

    _balance += amount;
}
```

} ``` The object protects its own business rules.

---

3. What is abstraction?

Expected answer: Abstraction means exposing what the consumer needs while hiding unnecessary implementation details. For example: `csharp id="z5l52d"`
`public interface IEmailService`
`{`
`    Task SendAsync(string email, string subject);`
`}`

The caller does not need to know whether the implementation uses SendGrid, SMTP, or another provider.

---

4. What is inheritance and when would you use it?

Expected answer:

Inheritance allows a derived class to reuse or extend behavior from a base class.

Use it when there is a genuine **is-a** relationship.

Example:

- `Dog` is an `Animal`
- `Cat` is an `Animal`

A strong candidate should also mention that inheritance should not be used just for code reuse. In many cases, composition is better.

---

5. What is polymorphism?

Expected answer:

Polymorphism allows different objects to be used through a common interface or base type.

Example:

```csharp id="xeg8l7" IEnumerable<IPaymentProcessor> processors = GetProcessors();

foreach (var processor in processors) { await processor.ProcessAsync(); }
```

```
Each processor can behave differently.

---

### 6. What is the difference between method overloading and overriding?

**Expected answer:**

Overloading means multiple methods have the same name but different parameters.

```csharp id="x7xq0m"
public int Add(int a, int b) => a + b;

public decimal Add(decimal a, decimal b) => a + b;
```

Overriding means a derived class changes the implementation of a virtual or abstract method.

---

7. What is the difference between an interface and an abstract class?

Expected answer:

Use an interface when you want to define a contract.

Use an abstract class when related classes need shared behavior or shared state.

A good answer should mention that the decision depends on the relationship between classes.

Intermediate

8. What do you mean by composition over inheritance?

Expected answer:
```

Instead of inheriting behavior from a base class, sometimes it is better to compose an object using smaller services.

For example:

Instead of:

```text id="0fq1q7" EmailNotification └──── BaseNotification

````
Use:

```text id="s4w9z2"
NotificationService
 ├── IEmailSender
 ├── ISmsSender
 └── IPushNotificationSender
````

This usually provides more flexibility and reduces tight coupling.

---

9. When can inheritance become difficult to maintain?

Expected answer:

Inheritance can become problematic when:

- The hierarchy becomes too deep.
- Child classes depend heavily on base-class implementation.
- A change in the base class affects many derived classes.
- Some child classes do not actually support all base behavior.

A strong candidate should suggest composition or strategy patterns when behavior varies independently.

---

10. What are `virtual`, `abstract`, and `sealed`?

Expected answer:

- `virtual`: Can be overridden.
- `abstract`: Must be implemented by a derived class.
- `sealed`: Prevents further inheritance or overriding.

The candidate should know practical use cases, not just definitions.

---

11. Suppose many classes inherit from one base class. What can happen if you change that base class?

Expected answer:

A change may unexpectedly affect all derived classes.

For example, changing validation logic in a base class may cause behavior changes in multiple child classes.

This is one reason deep inheritance hierarchies can become difficult to maintain.

---

12. What is the difference between association, aggregation, and composition?

Expected answer:

Association means objects know or work with each other.

Aggregation means one object contains or references another, but both can exist independently.

Composition means one object strongly owns another object's lifecycle.

The candidate does not need to use textbook-perfect terminology if the relationship is understood correctly.

---

13. What makes an object-oriented design easy to test?

Expected answer:

Good design usually has:

- Clear responsibilities.
- Small focused classes.
- Explicit dependencies.
- Minimal static/global state.
- Interfaces or other testable boundaries where useful.

Deep inheritance and hidden dependencies usually make testing harder.

---

14. If you have a very large `switch` statement based on provider type, how would you improve it?

Expected answer:

A common approach is Strategy Pattern.

Example:

```csharp id="8ib1lu" public interface IPaymentProvider { bool CanHandle(string provider); Task ProcessAsync(); }
```

```
Different providers implement the interface.

The application selects the appropriate implementation.

---

## Advanced

### 15. How would you design a payment system supporting multiple payment
providers?

**Expected answer:**

A good design might include:

- A common abstraction.
- Separate provider implementations.
- Dependency injection.
- Provider-specific adapters.
- Clear error handling.
- Idempotency.
- Logging and observability.

Example:

```csharp id="d4s7he"
public interface IPaymentProcessor
{
    Task<PaymentResult> ProcessAsync(
        PaymentRequest request,
        CancellationToken cancellationToken);
}
```
```

Each provider implements the same business contract.

---

16. What does “Tell, Don't Ask” mean?

Expected answer:

Instead of retrieving an object's data and applying its business rules somewhere else, ask the object to perform the behavior.

Less ideal:

```csharp id="7znnt9" if (order.Status == "Pending") { order.Status = "Cancelled"; }

```
Better:

```csharp id="cccnwk"
order.Cancel();
```

The business rule stays inside the object responsible for it.

---

17. How would poor OOP design affect a large application?

Expected answer:

It can cause:

- Tight coupling.
- Difficult testing.
- Unexpected side effects.
- Difficult changes.
- Duplicate business logic.
- Large classes with too many responsibilities.

A strong candidate should connect design quality with maintainability.

---

18. How do you know when a class has too many responsibilities?

Expected answer:

Warning signs include:

- Very large class.
- Many unrelated dependencies.
- Constructor with many services.
- Multiple reasons for the class to change.
- Difficult unit tests.

The solution is not automatically splitting every class. Responsibilities should be logically cohesive.

---

19. What would make you choose composition instead of inheritance?

Expected answer:

Choose composition when:

- Behavior can change independently.
- There is no strong is-a relationship.
- You need to combine different behaviors.
- Inheritance would create many special-case subclasses.

---

20. Give an example of an OOP design decision you would make differently today.

Expected answer:

This is a real-experience question.

Look for:

- Honest reflection.
- Understanding of trade-offs.
- Ability to explain why the original decision caused problems.
- A practical improvement.

---

2. SOLID Principles

Basic

1. What does SOLID stand for?

Expected answer:

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

The candidate should understand the concepts, not only expand the acronym.

---

2. What is Single Responsibility Principle?

Expected answer:

A class should have one clear responsibility or one main reason to change.

For example, this class may have too many responsibilities:

```text id="emrl6g" OrderService ├── Validate order ├── Save order ├── Send email ├── Generate PDF └── Calculate pricing

```
These responsibilities may change for different reasons.

---

3. Can a class have only one responsibility but many methods?

Expected answer:

Yes.

Single Responsibility does not mean one method per class.

A class can have many methods as long as they belong to one cohesive responsibility.

---

4. What is Dependency Inversion?

Expected answer:

High-level business logic should not directly depend on infrastructure details.

Instead of:

```csharp id="u5c6cv"
public class OrderService
{
    private readonly SqlOrderRepository _repository;
}
```

Depend on an abstraction:

```csharp id="15glw0" public class OrderService { private readonly IOrderRepository _repository; }
```

```
---

### 5. What is Interface Segregation Principle?

**Expected answer:**

A class should not be forced to implement methods it does not need.

Instead of one large interface:

```csharp id="wwc2gq"
IUserService
```

with many unrelated operations, create focused contracts based on actual needs.

---

6. What is Open/Closed Principle?

Expected answer:

The design should allow new behavior to be added without repeatedly changing stable existing code.

For example, instead of modifying a large switch statement every time a new payment provider is added, use separate provider implementations.

Intermediate

7. Give a real example of a class violating SRP.

Expected answer:

For example:

```text id="s9dp17"
UserService
├── User validation
├── Database access
├── Password hashing
├── Email sending
├── PDF generation
└── Report generation
```

```
This is difficult to maintain and test.

---

### 8. Does using interfaces everywhere mean your application follows SOLID?

**Expected answer:**
```
```

No.

Creating interfaces unnecessarily can make the code more complicated.

An abstraction should solve a real problem such as:

- Multiple implementations.
- A clear architectural boundary.
- Easier testing.
- Expected variation.

---

9. Can Open/Closed Principle be overused?

Expected answer:

Yes.

Creating complex extension systems for code that will never change is unnecessary.

A strong candidate should understand the balance between flexibility and simplicity.

---

10. What happens if a Singleton depends directly on a Scoped service?

Expected answer:

This creates a lifetime problem.

For example, a scoped database context normally belongs to one request.

A singleton lives for the lifetime of the application and should not hold onto that scoped dependency.

A common solution in a background service is to create a scope using `IServiceScopeFactory`.

---

11. How does Dependency Injection relate to Dependency Inversion?

Expected answer:

```
Dependency Inversion is a design principle.

Dependency Injection is one technique used to provide dependencies.

DI helps us implement DIP, but simply using DI does not automatically mean the
design follows DIP.

---

### 12. How do SOLID principles help testing?

**Expected answer:**

Focused responsibilities and explicit dependencies make code easier to test.

For example:

```csharp id="nygf1c"
public class OrderService
{
    private readonly IPaymentService _paymentService;
}
```
```

The payment dependency can be controlled during testing.

---

13. What is a sign that a service may have too many dependencies?

Expected answer:

A constructor like this may indicate a design problem:

```csharp id="x2n2b8" public OrderService( IRepository repository, IEmailService emailService, IPaymentService paymentService, ICache cache, ILogger logger, IReportService reportService, IFileService fileService)

```
It does not automatically mean the class is wrong, but it should be reviewed for
too many responsibilities.

Advanced

14. How would you apply SOLID principles to a legacy application?
```
```

````
**Expected answer:**

Do not rewrite everything.

A practical approach:

1. Identify the area being changed.
2. Add tests around important existing behavior.
3. Create small boundaries around dependencies.
4. Extract responsibilities gradually.
5. Avoid unnecessary large refactoring.

A strong senior developer should prefer incremental improvement.

---

### 15. Suppose your business logic directly uses MongoDB or SQL classes
everywhere. What problem can that create?

**Expected answer:**

Business logic becomes tightly coupled to the database implementation.

Database changes become harder.

Testing can also become more difficult.

A candidate should discuss introducing appropriate abstractions at meaningful
boundaries rather than blindly creating repository interfaces for every table.

---

### 16. How would you design an extensible notification system?

**Expected answer:**

Possible channels:

- Email.
- SMS.
- Push notification.

A good design could use:

```csharp id="n5k2ac"
public interface INotificationSender
{
````
````

```
    Task SendAsync(Notification notification);
}
```

Different implementations handle different channels.

---

17. When does too much abstraction become a problem?

Expected answer:

When the code becomes harder to understand than the original problem.

Examples:

- Interface for every class.
- Factory for objects that never vary.
- Multiple layers with no meaningful responsibility.

---

18. What is more important: strictly following SOLID or keeping the solution simple?

Expected answer:

A strong answer is balance.

SOLID principles are guidelines that help design maintainable systems. They should not be applied mechanically.

---

19. How would you review whether a new abstraction is actually useful?

Expected answer:

Ask:

- What problem does it solve?
- Is there real variation?
- Who owns the contract?
- Does it reduce coupling?
- Does it make testing easier?
- Does it add unnecessary complexity?

---

20. Tell me about a design you improved because it was difficult to maintain.

Expected answer:

Look for real experience and practical reasoning.

The candidate should explain:

- Original problem.
- Why the design was difficult.
- Changes made.
- Trade-offs.
- Result.

---

3. SQL

Basic

1. What is the difference between `INNER JOIN` and `LEFT JOIN`?

Expected answer:

`INNER JOIN` returns only matching records.

`LEFT JOIN` returns all records from the left table and matching records from the right table.

---

2. What is a primary key?

Expected answer:

A primary key uniquely identifies a row.

It should not contain duplicate or null values.

---

3. What is the difference between `WHERE` and `HAVING`?

Expected answer:

`WHERE` filters rows before grouping.

`HAVING` filters groups after aggregation.

Example:

```sql id="e84cx3" SELECT DepartmentId, COUNT() FROM Employees WHERE IsActive = 1 GROUP BY DepartmentId HAVING COUNT() > 10;
```

```
---

### 4. What is an index?

**Expected answer:**

An index helps the database find data more efficiently.

However, indexes also have costs:

- Storage.
- Slower inserts.
- Slower updates.
- Maintenance overhead.

A strong candidate should not say “more indexes are always better.”

---

### 5. What is normalization?

**Expected answer:**

Normalization reduces unnecessary duplicate data and helps prevent data
inconsistency.

However, some systems intentionally denormalize data for reporting or
performance.

---

### 6. What is the difference between a clustered and non-clustered index in SQL
Server?

**Expected answer:**

A clustered index determines how the table's data is organized at the leaf
level.

A non-clustered index is a separate structure containing indexed keys and
references to the underlying data.
```

```
---

## Intermediate

### 7. A query is slow. What steps would you take to investigate it?

**Expected answer:**

A strong process:

1. Get the actual query.
2. Check execution plan.
3. Look at estimated versus actual rows.
4. Check indexes.
5. Check blocking.
6. Check statistics.
7. Review query predicates.
8. Test improvements.

Do not immediately add an index without understanding the problem.

---

### 8. When a SQL query is slow, how do you use the execution plan to find the problem?

**Expected answer:**

Look for things such as:

- Large table scans.
- Expensive operators.
- Key lookups.
- Incorrect row estimates.
- Sort operations.
- Missing or unsuitable indexes.
- Spills.

The candidate should understand that an execution plan is evidence, not an automatic answer.

---

### 9. What is a composite index and why does column order matter?

**Expected answer:**

A composite index contains multiple columns.
```

```
The order matters because SQL Server can efficiently use the leading columns for
many query patterns.

Example:

```text id="it4m0w"
(BuildingId, Status)
```

may work differently from:

```text id="1vq7h9" (Status, BuildingId)

```
depending on the query and data distribution.

---

10. What is a covering index?

Expected answer:

A covering index contains the data required by a query so the database may not
need additional lookups.

Example:

If a query needs:

```text id="pr81zx"
SELECT Name
FROM Users
WHERE BuildingId = 10
```

an index containing `BuildingId` and `Name` may cover the query.

---

11. What is a deadlock?

Expected answer:

A deadlock happens when transactions wait for each other in a cycle.

The database chooses one transaction as a victim.
```

Ways to reduce deadlocks:

- Keep transactions short.
- Access resources in a consistent order.
- Improve indexes.
- Retry deadlock victims where appropriate.

---

12. What are transaction isolation levels?

Expected answer:

Isolation levels control how transactions see changes made by other transactions.

The candidate should understand problems such as:

- Dirty reads.
- Non-repeatable reads.
- Phantom reads.

---

13. What is the difference between optimistic and pessimistic concurrency?

Expected answer:

Suppose two users edit the same record.

Optimistic concurrency allows both to read, but detects conflict when updating.

Pessimistic concurrency locks data earlier to reduce concurrent modification.

A good answer should discuss trade-offs.

Advanced

14. A query is fast for one parameter but very slow for another. Why might that happen?

Expected answer:

One possible reason in SQL Server is parameter sniffing.

SQL Server may reuse a cached execution plan created for a different parameter value.

If data distribution is very different, that plan may not perform well for another value.

Possible solutions depend on the actual problem and may include:

- Query changes.
- Updated statistics.
- Recompilation.
- Query Store.
- Specific hints where justified.

---

15. How would you handle pagination for millions of rows?

Expected answer:

Large `OFFSET` values can become inefficient.

Keyset or seek pagination can be better.

Example:

```sql id="4x7m4a"
SELECT TOP 50 *
FROM Orders
WHERE Id > @LastId
ORDER BY Id;
```

```
This requires a stable and suitable ordering strategy.

---

### 16. How would you investigate database blocking?

**Expected answer:**

Look at:

- Active sessions.
- Blocking chains.
- Long-running transactions.
- Query execution.
- Indexes.
- Transaction scope.

The candidate should distinguish blocking from deadlocks.

---

### 17. When should you avoid adding another index?
```

Expected answer:

Avoid adding indexes blindly.

Consider:

- Existing indexes.
- Query frequency.
- Write overhead.
- Storage.
- Selectivity.
- Actual execution plan.

---

18. How would you design database changes for a production application with zero or minimal downtime?

Expected answer:

Prefer backward-compatible changes.

For example:

1. Add a new nullable column.
2. Deploy application supporting both old and new versions.
3. Migrate data.
4. Remove old behavior later.

Avoid deployment steps that immediately break the currently running application.

---

19. How would you prevent duplicate data when multiple requests arrive at the same time?

Expected answer:

Possible approaches:

- Unique constraints.
- Transactions.
- Idempotency keys.
- Appropriate concurrency handling.

Database constraints are important because application-level checks alone can have race conditions.

```
---

### 20. Describe the most difficult SQL performance problem you have solved.

**Expected answer:**

Look for a structured investigation rather than memorized answers.

---

# 4. C#

## Basic

### 1. What is the difference between value types and reference types?

**Expected answer:**

Value types contain their value.

Reference types refer to an object.

Examples:

```csharp id="a5k9ep"
int age = 30;
```

`int` is a value type.

```csharp id="j3n67z" Customer customer = new Customer();

```
`Customer` is a reference type.

Avoid accepting the oversimplified answer that value types always live on the
stack and reference types always live on the heap.

---

2. What is boxing and unboxing?

Expected answer:

Boxing converts a value type into an object representation.

Unboxing converts it back.
```

```csharp id="2g93fu"
int number = 10;

object value = number; // Boxing

int result = (int)value; // Unboxing
```

Excessive boxing can create unnecessary allocations.

---

3. What is the difference between `string` and `StringBuilder`?

Expected answer:

`string` is immutable.

If many concatenations happen repeatedly, `StringBuilder` can reduce unnecessary object creation.

---

4. What is the difference between `const` and `readonly`?

Expected answer:

`const` is a compile-time constant.

`readonly` can be assigned during declaration or construction and cannot be changed afterward.

---

5. What are `ref`, `out`, and `in`?

Expected answer:

- `ref`: Pass by reference and read/write.
- `out`: Method must assign a value.
- `in`: Pass by readonly reference.

The candidate should know when they are useful rather than forcing unnecessary usage.

---

6. What are nullable reference types?

Expected answer:

Nullable reference types help the compiler warn about possible null problems.

Example:

```csharp id="ty8fsv"
string? name = GetName();
```

```
This does not prevent null at runtime. It provides compiler analysis.

---

## Intermediate

### 7. How does `async` and `await` work?

**Expected answer:**

`async` and `await` simplify asynchronous programming.

For I/O operations such as:

- HTTP calls.
- Database calls.
- File operations.

`await` allows the operation to wait asynchronously without unnecessarily
blocking a thread.

Avoid:

```csharp id="n2obup"
var result = service.GetDataAsync().Result;
```
```

Prefer:

```csharp id="0wvj11"
var result = await service.GetDataAsync();
```

```
A strong candidate should know that `async` does not automatically mean a new
thread.

---

### 8. What is the difference between `Task` and `Thread`?

**Expected answer:**

A `Thread` is an operating system execution thread.
```

````
A `Task` represents asynchronous or concurrent work.

A task does not necessarily mean a dedicated thread.

---

### 9. What is `CancellationToken` and how do you use it?

**Expected answer:**

It allows an operation to be cancelled cooperatively.

Example:

```csharp id="w4r0d2"
public async Task ProcessAsync(
    CancellationToken cancellationToken)
{
    await _service.ProcessAsync(cancellationToken);
}
````

A good candidate should propagate the token to lower-level operations where supported.

---

10. What is the difference between `IEnumerable` and `IQueryable`?

Expected answer:

`IEnumerable` usually works with objects in memory.

`IQueryable` builds expressions that may be translated by providers such as Entity Framework into SQL.

A candidate should understand the risk of bringing too much data into memory.

---

11. What is deferred execution?

Expected answer:

Some LINQ queries do not execute when declared.

They execute when enumerated.

Example:
````

```csharp id="u9r8ee" var activeUsers = users.Where(x => x.IsActive);

```
The query may execute when iterating or calling `ToList()`.

---

12. What is `IDisposable` used for?

Expected answer:

It allows deterministic cleanup of resources.

Example:

```csharp id="xzm3l9"
using var stream = File.OpenRead(path);
```

The stream is disposed when it is no longer needed.

---

13. What are delegates and events?

Expected answer:

Delegates represent methods that can be passed around.

Events are commonly used to notify subscribers while controlling who can raise the event.

Advanced

14. What can cause a memory leak in a .NET application even though .NET has garbage collection?

Expected answer:

Garbage collection only removes objects that are no longer reachable.

Memory can continue growing because of:

- Static collections.
- Event subscriptions.
- Long-lived caches.
- Captured references.

- Objects kept in memory unintentionally.
- Unmanaged resources not being disposed.

---

15. What is thread pool starvation?

Expected answer:

It can happen when many thread-pool threads are blocked.

For example:

```csharp id="k12h9c" var result = service.GetDataAsync().Result;
```

```
Under heavy load, blocked threads can reduce the application's ability to
process new work.

---

### 16. What is the difference between `Task.WhenAll` and controlled
concurrency?

**Expected answer:**

`Task.WhenAll` waits for all supplied tasks.

If you create thousands of expensive tasks simultaneously, this can overload:

- The application.
- Database.
- External API.

For large workloads, bounded concurrency may be safer.

---

### 17. When would you use `Span<T>` or `Memory<T>`?

**Expected answer:**

They are advanced performance features useful for reducing allocations and
efficiently working with slices of memory.

Do not expect every 6+ year developer to use them daily.

A strong candidate should know when performance profiling justifies using them.
```

```
---

### 18. What are records and when would you use them?

**Expected answer:**

Records are useful for data-focused models and value-like equality semantics.

They are often useful for:

- DTOs.
- Messages.
- Immutable models.

Be careful using mutable records for entities with complex lifecycle behavior.

---

### 19. How would you investigate high memory usage in production?

**Expected answer:**

A good approach:

1. Confirm memory growth.
2. Check GC metrics.
3. Take memory dumps if appropriate.
4. Use profiling tools.
5. Identify retained objects.
6. Check caches, static references, event subscriptions, and large collections.

---

### 20. How would you process thousands of independent tasks safely?

**Expected answer:**

Do not automatically start everything simultaneously.

Consider:

- Bounded concurrency.
- Cancellation.
- Retry policy.
- Error handling.
- Rate limits.
- Memory usage.
```

````
---

# 5. LINQ

## Basic

### 1. What is LINQ?

**Expected answer:**

LINQ provides a common way to query data.

It can work with:

- Collections.
- Arrays.
- Entity Framework queries.
- Other providers.

---

### 2. What is the difference between `Where` and `Select`?

**Expected answer:**

`Where` filters data.

`Select` transforms or projects data.

```csharp id="3fv18m"
var names = users
    .Where(x => x.IsActive)
    .Select(x => x.Name);
````

---

3. What is the difference between `Select` and `SelectMany`?

Expected answer:

`SelectMany` is used to flatten nested collections.

Example:

```csharp id="me9x0a" var orders = customers.SelectMany(x => x.Orders);

````
---

4. What is the difference between `First`, `FirstOrDefault`, `Single`, and
`SingleOrDefault`?

Expected answer:

- `First`: Requires at least one.
- `FirstOrDefault`: Returns default when none exists.
- `Single`: Requires exactly one.
- `SingleOrDefault`: Allows zero but throws when multiple exist.

The choice should match the business expectation.

---

5. What does `GroupBy` do?

Expected answer:

It groups items based on a key.

Example:

```csharp id="2qzg8z"
var groups = employees.GroupBy(x => x.DepartmentId);
````

---

6. What is the difference between `OrderBy` and `ThenBy`?

Expected answer:

`OrderBy` defines the primary sorting.

`ThenBy` adds secondary sorting.

Intermediate

7. What problems can happen if a LINQ query is enumerated multiple times?

Expected answer:

If the query is expensive or calls a database/provider, it may execute multiple times.
````

Example:

```csharp id="trqj9l" var query = users.Where(x => x.IsActive);

var count = query.Count(); var list = query.ToList();
```

```
Depending on the source, work may happen twice.

Materialize intentionally when appropriate.

---

### 8. Why is `Any()` often better than `Count() > 0`?

**Expected answer:**

When checking existence, `Any()` communicates the intent and may stop as soon as
a matching item is found.

---

### 9. What is the difference between `IQueryable` and `IEnumerable` when using
Entity Framework?

**Expected answer:**

`IQueryable` may translate operations into SQL.

After switching to in-memory enumeration, later operations happen in application
memory.

The candidate should understand when data is actually retrieved.

---

### 10. What is `ToList()` doing?

**Expected answer:**

It materializes the query into a list.

This is sometimes necessary but should not be called unnecessarily early.

---

### 11. How do LINQ joins work?
```

Expected answer:

LINQ supports joins, including:

- Inner joins.
- Group joins.
- Left join patterns.

The candidate should understand when joins are translated to SQL versus performed in memory.

---

12. What is an expression tree?

Expected answer:

A delegate executes code.

An expression tree represents code as data.

This allows providers such as Entity Framework to inspect and translate expressions into SQL.

---

13. What is `AsEnumerable()` and when would you use it?

Expected answer:

It switches the following operations to LINQ-to-Objects.

Use it intentionally.

Doing it too early can cause a large amount of database data to be loaded into memory.

Advanced

14. Why might a LINQ query work with a `List<T>` but fail with Entity Framework?

Expected answer:

A `List<T>` executes normal .NET code.

Entity Framework needs to translate many operations into SQL.

A custom .NET method may not be translatable.

---

15. What is the N+1 query problem?

Expected answer:

It happens when one query loads a list, then additional queries are executed for each item.

Example:

1 query for 100 orders.

Then 100 queries for related customer data.

A strong candidate should discuss:

- Projection.
- Includes when appropriate.
- Joins.
- Batch loading.

---

16. How would you optimize a slow LINQ operation in memory?

Expected answer:

First understand complexity.

Possible improvements:

- Avoid repeated enumeration.
- Avoid nested loops.
- Use `Dictionary` or `HashSet` for lookups.
- Avoid unnecessary materialization.

---

17. How would you review a complex LINQ query?

Expected answer:

````
Check:

1. Correctness.
2. Readability.
3. Execution location.
4. Generated SQL if applicable.
5. Number of database calls.
6. Materialization.
7. Performance.

---

### 18. When would a compiled query be useful?

**Expected answer:**

For demonstrated high-frequency query paths where query compilation overhead
matters.

Measure before optimizing.

---

### 19. What problems can closures cause in LINQ?

**Expected answer:**

Variables captured by a query may change before the query is executed.

Because of deferred execution, this can produce unexpected results.

---

### 20. How would you avoid loading unnecessary columns from the database?

**Expected answer:**

Use projection.

Example:

```csharp id="nn95yf"
var users = await context.Users
    .Where(x => x.IsActive)
    .Select(x => new UserDto
    {
        Id = x.Id,
        Name = x.Name
````
````

```
    })
    .ToListAsync();
```

---

6. .NET / ASP.NET Core

Basic

1. Explain Transient, Scoped, and Singleton services.

Expected answer:

- Transient: New instance when resolved.
- Scoped: Usually one instance per HTTP request.
- Singleton: One instance for the application lifetime.

The candidate should understand lifetime compatibility.

---

2. What is middleware?

Expected answer:

Middleware processes HTTP requests and responses.

Examples:

- Exception handling.
- Authentication.
- Logging.
- CORS.

---

3. Why does middleware order matter?

Expected answer:

Middleware executes as a pipeline.

For example, authentication must happen before authorization.

Incorrect ordering can cause incorrect behavior.

---

4. How does configuration work in ASP.NET Core?

Expected answer:

Configuration can come from:

- JSON files.
- Environment variables.
- Secret stores.
- Command line.

Environment-specific values can override base configuration.

---

5. What is the difference between authentication and authorization?

Expected answer:

Authentication answers:

> Who are you?

Authorization answers:

> Are you allowed to perform this action?

---

6. What is Kestrel?

Expected answer:

Kestrel is the cross-platform web server used by ASP.NET Core.

It can run directly or behind a reverse proxy.

Intermediate

7. How would you handle exceptions globally in an ASP.NET Core API?

Expected answer:

Use centralized exception handling.

The solution should:

- Return appropriate status codes.
- Log useful context.
- Avoid exposing internal exception details.
- Return consistent error responses.

---

8. What is `BackgroundService` used for?

Expected answer:

It is used for long-running background work managed by the application host.

Examples:

- Queue processing.
- Scheduled processing.
- Polling.

---

9. What are health checks?

Expected answer:

Health checks expose whether an application or dependency is healthy.

They may be used by:

- Load balancers.
- Kubernetes.
- Monitoring systems.

---

10. What is the Options Pattern?

Expected answer:

It maps configuration to strongly typed classes.

Example:

```csharp id="s70l7m" public class EmailOptions { public string ApiKey { get; set; } }
```

```
This improves configuration organization and validation.

---

### 11. What is `IServiceScopeFactory` used for?

**Expected answer:**

It allows explicit creation of dependency injection scopes.

A common example is a singleton background service needing scoped services such as repositories or database contexts.

---

### 12. How would you validate API input?

**Expected answer:**

Use:

- Model validation.
- Data annotations or FluentValidation.
- Business validation.

Do not assume client-side validation is sufficient.

---

### 13. What is the difference between Minimal APIs and Controllers?

**Expected answer:**

Minimal APIs are concise and useful for simpler endpoints.

Controllers provide more structured organization and conventions.

The correct choice depends on application complexity and team preferences.

---

## Advanced

### 14. How would you design a reliable background queue processor?

**Expected answer:**
```

````
Consider:

- Cancellation.
- Bounded concurrency.
- Retry logic.
- Error classification.
- Idempotency.
- Logging.
- Monitoring.
- Durable queue/state when required.

A strong candidate should understand that simply wrapping work in `Task.Run()`
is not a reliable background processing architecture.

---

### 15. How would you prevent thread pool starvation in an API?

**Expected answer:**

Avoid blocking async code with:

- `.Result`
- `.Wait()`

Use true asynchronous APIs for I/O.

Control expensive parallel operations.

---

### 16. How would you manage application secrets?

**Expected answer:**

Do not hardcode secrets.

Use:

- Secret stores.
- Managed/workload identity.
- Least privilege.
- Rotation.

---

### 17. How would you version an API?
````

```
**Expected answer:**

The candidate should discuss:

- Backward compatibility.
- Versioning strategy.
- Deprecation.
- Client impact.

Avoid unnecessary versions for simple additive changes.

---

### 18. What production monitoring would you add to an API?

**Expected answer:**

At minimum:

- Structured logs.
- Metrics.
- Request duration.
- Error rate.
- Dependency failures.
- Distributed traces.
- Health checks.
- Alerts.

---

### 19. How would you investigate a slow API endpoint?

**Expected answer:**

Start with evidence:

1. Identify slow requests.
2. Check traces.
3. Find slow dependencies.
4. Check database queries.
5. Check CPU/memory.
6. Check external calls.
7. Reproduce if possible.

---

### 20. How would you deploy an API without breaking existing clients?
```

Expected answer:

Use backward-compatible changes and deployment strategies that allow old and new versions to coexist temporarily.

---

7. Microservices

Basic

1. What is a microservice?

Expected answer:

A microservice is an independently deployable service responsible for a specific business capability.

The important point is not simply making many small APIs.

---

2. What is the difference between a monolith and microservices?

Expected answer:

A monolith is usually simpler to build and operate.

Microservices provide independent deployment and scaling but introduce distributed-system complexity.

A strong candidate should not automatically claim microservices are always better.

---

3. When would you use synchronous communication and when would you use messaging?

Expected answer:

Synchronous communication is useful when an immediate response is required.

Messaging is useful when work can happen asynchronously and services should be decoupled.

```
### 4. What is an API Gateway?

**Expected answer:**

It can provide:

- Routing.
- Authentication.
- Rate limiting.
- Aggregation.
- Cross-cutting policies.

---

### 5. Should every microservice have its own database?

**Expected answer:**

Services should own their data boundaries.

Other services should not directly depend on internal tables.

Physical infrastructure sharing may be possible, but tight data coupling should
be avoided.

---

## Intermediate

### 6. What is eventual consistency?

**Expected answer:**

Different services may temporarily have different views of the data.

They eventually converge through asynchronous processing.

The candidate should explain how the business handles the temporary state.

---

### 7. What is idempotency?

**Expected answer:**

If the same operation is processed more than once, it should not create
unintended duplicate effects.
```

Example:

A payment request with the same idempotency key should not charge the customer twice.

---

8. How would you make sure that a database update and an event are not lost if the application crashes between the two operations?

Expected answer:

This is a common reason for the Outbox Pattern.

The application saves:

- Business data.
- Event information.

in the same database transaction.

A separate process later publishes the event reliably.

---

9. Suppose one business operation updates multiple microservices. Step 3 fails after steps 1 and 2 succeeded. How would you handle it?

Expected answer:

This is a distributed transaction problem.

A Saga-style approach can use:

- Local transactions.
- Compensating actions.
- Orchestration or choreography.

The candidate should understand that traditional database transactions cannot simply span independent services without trade-offs.

---

10. How do you handle duplicate messages?

Expected answer:

Assume duplicates can happen.

Use:

- Message IDs.
- Idempotency records.
- Safe business operations.

---

11. What is a circuit breaker and when would you use it?

Expected answer:

If a dependency is repeatedly failing, a circuit breaker can temporarily stop calls instead of continuing to overload an unhealthy service.

---

12. Why are timeouts important in microservices?

Expected answer:

Without timeouts, requests may wait too long and consume resources.

Timeouts help prevent cascading failures.

---

13. What is service discovery?

Expected answer:

A mechanism for locating service instances dynamically.

In modern cloud environments, DNS or orchestration platforms may provide this.

Advanced

14. How would you design reliable event processing?

Expected answer:

Consider:

- Durable messaging.
- Outbox.

```
- Idempotent consumers.
- Retry strategy.
- Dead-letter queues.
- Monitoring.
- Schema compatibility.

---

### 15. How would you trace one request across multiple services?

**Expected answer:**

Use correlation and distributed tracing.

The same trace context should flow across services.

Logs, metrics, and traces should make it possible to follow the request.

---

### 16. What mistakes would you avoid when building microservices?

**Expected answer:**

Common problems:

- Too many services too early.
- Shared database coupling.
- Too many synchronous calls.
- Distributed monolith.
- Unclear ownership.
- No observability.

---

### 17. How would you handle partial failure?

**Expected answer:**

Consider:

- Timeouts.
- Safe retries.
- Circuit breakers.
- Queues.
- Compensation.
- Clear application states.
```

```
---

### 18. How would you decide service boundaries?

**Expected answer:**

Use:

- Business capabilities.
- Domain boundaries.
- Data ownership.
- Change patterns.
- Team ownership.

Do not split services only by technical layers.

---

### 19. When would you choose a modular monolith instead of microservices?

**Expected answer:**

When:

- The system is not large enough to justify distributed complexity.
- Boundaries are still evolving.
- The team cannot support operational overhead.

---

### 20. Describe a distributed-system problem you have personally handled.

**Expected answer:**

Look for real understanding of:

- Failures.
- Retries.
- Duplicate processing.
- Observability.
- Data consistency.

---

# 8. Cloud

## Basic
```

```
### 1. What is the difference between IaaS, PaaS, and SaaS?

**Expected answer:**

- IaaS: Infrastructure such as VMs.
- PaaS: Managed application platforms.
- SaaS: Complete software used by customers.

---

### 2. What is horizontal and vertical scaling?

**Expected answer:**

Horizontal scaling adds more instances.

Vertical scaling increases resources of an existing instance.

---

### 3. What is autoscaling?

**Expected answer:**

Automatically increasing or decreasing capacity based on metrics, schedules, or
rules.

---

### 4. What is a managed identity or workload identity?

**Expected answer:**

It allows an application to authenticate to cloud resources without storing
long-lived credentials in application code.

---

### 5. What is a load balancer?

**Expected answer:**

It distributes requests between healthy application instances.

---

## Intermediate
```

```
### 6. How should you manage secrets in a cloud application?

**Expected answer:**

Use:

- Managed secret stores.
- Application identity.
- Least privilege.
- Rotation.

Do not store secrets directly in source code.

---

### 7. What is the principle of least privilege?

**Expected answer:**

Give an application or user only the permissions required to perform its task.

---

### 8. What is Infrastructure as Code?

**Expected answer:**

Cloud infrastructure is defined in version-controlled configuration so
environments can be created consistently and repeatably.

---

### 9. What is the difference between High Availability and Disaster Recovery?

**Expected answer:**

High Availability focuses on reducing interruption from failures.

Disaster Recovery focuses on recovering after larger incidents.

---

### 10. What are RTO and RPO?

**Expected answer:**

RTO is the acceptable recovery time.
```

```
RPO is the acceptable amount of data loss measured in time.

---

### 11. What is a private endpoint?

**Expected answer:**

It allows private network connectivity to supported cloud resources instead of
using a public endpoint.

---

### 12. How would you monitor a cloud application?

**Expected answer:**

Use:

- Logs.
- Metrics.
- Traces.
- Alerts.
- Health checks.
- Dashboards.

---

## Advanced

### 13. How would you design an application for high availability?

**Expected answer:**

Avoid single points of failure.

Depending on requirements:

- Multiple instances.
- Multiple zones.
- Redundant dependencies.
- Failover strategy.
- Tested recovery.

---

### 14. How would you deploy with minimal downtime?
```

Expected answer:

Possible strategies:

- Rolling deployment.
- Blue-green deployment.
- Canary deployment.

The candidate should discuss rollback and backward compatibility.

---

15. How do you control cloud costs?

Expected answer:

- Right-size resources.
- Autoscale.
- Remove unused resources.
- Use budgets.
- Monitor cost.
- Review storage lifecycle.

---

16. What is the shared responsibility model?

Expected answer:

Cloud providers manage some parts of the infrastructure, but customers still have security and configuration responsibilities.

Responsibilities depend on the service model.

---

17. How would you secure service-to-service communication?

Expected answer:

Use:

- Identity-based authentication.
- TLS.
- Least privilege.
- Private networking where appropriate.
- Token validation.

```
---

### 18. What is the difference between monitoring and observability?

**Expected answer:**

Monitoring helps detect known problems using predefined metrics and alerts.

Observability helps investigate unexpected problems using logs, metrics, and
traces.

---

### 19. How would you investigate a cloud-only performance problem?

**Expected answer:**

Compare:

- Application instances.
- Resource usage.
- Scaling.
- Dependencies.
- Network behavior.
- Region/environment configuration.

Use telemetry to correlate evidence.

---

### 20. Describe a production cloud issue you have investigated.

**Expected answer:**

Look for a structured debugging process and ownership.

---

# 9. AI Tools

## Basic

### 1. How do you currently use AI tools as a developer?

**Expected answer:**

Possible uses:
```

```
- Understanding code.
- Generating boilerplate.
- Refactoring.
- Writing tests.
- Code review assistance.
- Documentation.

Look for practical experience.

---

### 2. Can you trust AI-generated code without reviewing it?

**Expected answer:**

No.

Review:

- Correctness.
- Security.
- Performance.
- Architecture.
- Tests.

---

### 3. What is an AI hallucination?

**Expected answer:**

When an AI produces information or code that appears confident but is incorrect
or invented.

---

### 4. What information should you avoid sharing with an AI tool?

**Expected answer:**

Depending on company policy:

- Secrets.
- Credentials.
- Customer data.
- Proprietary code.
- Sensitive information.
```

```
---

### 5. What makes a good coding prompt?

**Expected answer:**

Include:

- Goal.
- Context.
- Constraints.
- Technology version.
- Expected output.
- Acceptance criteria.

---

## Intermediate

### 6. How would you use AI to understand a large legacy project?

**Expected answer:**

Do not give random code and blindly trust the output.

A good process:

1. Start with architecture.
2. Provide focused files.
3. Ask about data flow.
4. Verify against code.
5. Continue in smaller areas.

---

### 7. How can AI help with code review?

**Expected answer:**

It can help identify:

- Potential bugs.
- Missing tests.
- Edge cases.
- Security issues.
- Inconsistent patterns.

Human review remains necessary.
```

```
---

### 8. How would you validate AI-generated unit tests?

**Expected answer:**

Check whether:

- Tests verify meaningful behavior.
- Assertions can actually fail.
- Important edge cases exist.
- Tests are not only mocking implementation details.

---

### 9. What happens when an AI model does not have enough context about a large
project?

**Expected answer:**

It may:

- Make assumptions.
- Miss dependencies.
- Suggest incorrect changes.

Use focused context, project instructions, retrieval, and verification.

---

### 10. What is RAG?

**Expected answer:**

Retrieval-Augmented Generation retrieves relevant information and provides it to
the model as context before generating an answer.

---

### 11. How would you prevent an AI coding agent from changing unrelated code?

**Expected answer:**

Use:

- Clear scope.
- Small tasks.
```

```
- Repository instructions.
- Diff review.
- Automated tests.
- Explicit approval for broad changes.

---

### 12. What risks come with using AI for software development?

**Expected answer:**

- Incorrect code.
- Data leakage.
- Security vulnerabilities.
- Prompt injection.
- Dependency/license issues.
- Overreliance.

---

## Advanced

### 13. How would you build an AI-assisted code review process?

**Expected answer:**

A good process may include:

1. Retrieve relevant requirements.
2. Analyze changed files.
3. Check architecture rules.
4. Run tests/static analysis.
5. Produce structured findings.
6. Require human approval.

---

### 14. If an AI application reads documents or user input, how could malicious content try to influence the AI's behavior?

**Expected answer:**

This is related to prompt injection.

Untrusted content might contain instructions attempting to manipulate the AI.

Mitigation includes:
```

```
- Clear separation of trusted and untrusted content.
- Limited tool permissions.
- Validation.
- Human approval for sensitive actions.

---

### 15. How would you safely allow an AI agent to work with production systems?

**Expected answer:**

Default to:

- Read-only access.
- Least privilege.
- Scoped permissions.
- Approval gates.
- Audit logs.

Avoid giving unrestricted destructive access.

---

### 16. How can AI help during a production incident?

**Expected answer:**

It can:

- Summarize logs.
- Identify patterns.
- Suggest queries.
- Correlate events.

Humans should verify conclusions before taking action.

---

### 17. What is the difference between an AI assistant and an AI agent?

**Expected answer:**

An assistant primarily responds to requests.

An agent may plan steps and invoke tools/actions within its granted permissions.

---
```

18. How would you review an AI-generated architecture proposal?

Expected answer:

Check:

- Requirements.
- Assumptions.
- Security.
- Scalability.
- Failure scenarios.
- Cost.
- Operational complexity.

---

19. How would you measure whether AI is actually improving developer productivity?

Expected answer:

Do not measure generated lines of code.

Consider:

- Delivery time.
- Defects.
- Review rework.
- Developer effort.
- Quality.

---

20. What is responsible use of AI in software development?

Expected answer:

It includes:

- Protecting data.
- Validating output.
- Maintaining human accountability.
- Following company policy.
- Considering security and compliance.

---

10. Angular / React

```
## Basic

### 1. What is component-based development?

**Expected answer:**

The UI is divided into reusable components.

Each component should have a clear responsibility.

---

### 2. What is the difference between component state and input/props?

**Expected answer:**

Inputs/props are values provided to a component.

State is data managed by the component or application.

---

### 3. Why are stable keys important when rendering lists in React?

**Expected answer:**

Keys help React identify which items changed.

Using unstable keys can cause incorrect component reuse.

---

### 4. What is Angular Dependency Injection?

**Expected answer:**

Angular manages and provides dependencies using its injector and provider
system.

---

### 5. What is client-side routing?

**Expected answer:**

The application maps URLs to components/views without necessarily performing a
full browser page reload.
```

```
---

## Intermediate

### 6. What are Observables used for in Angular?

**Expected answer:**

Observables represent asynchronous streams.

They are commonly used for:

- HTTP.
- Events.
- Reactive forms.

---

### 7. What is the difference between a Promise and an Observable?

**Expected answer:**

A Promise represents one eventual result.

An Observable can produce multiple values over time and supports operators and
cancellation/unsubscription patterns.

---

### 8. What is `useEffect` used for in React?

**Expected answer:**

It is commonly used to synchronize a component with something outside React.

Examples:

- API subscriptions.
- Event listeners.
- Timers.

---

### 9. How do frontend memory leaks happen?

**Expected answer:**
```

```
Common causes:

- Subscriptions not cleaned up.
- Event listeners.
- Timers.
- References to old components.

---

### 10. What is lazy loading?

**Expected answer:**

Loading code only when a feature is needed instead of loading the entire
application initially.

---

### 11. How would you manage shared state?

**Expected answer:**

Keep state local when possible.

Use services, context, stores, or state-management tools when multiple areas
need coordinated state.

---

## Advanced

### 12. How does Angular know that data has changed and the UI needs to update?

**Expected answer:**

Angular uses change detection mechanisms.

A strong candidate may discuss:

- Change detection strategies.
- Immutable data patterns.
- Signals in modern Angular.

---

### 13. How would you optimize a slow frontend application?

**Expected answer:**
```

```
Profile first.

Possible areas:

- Bundle size.
- Large lists.
- Unnecessary rendering.
- Network calls.
- Caching.
- Lazy loading.

---

### 14. How do you prevent race conditions when multiple API requests are
running?

**Expected answer:**

Possible techniques:

- Cancel stale requests.
- Track request identity.
- Ignore outdated responses.
- Use appropriate RxJS operators.

---

### 15. How would you structure a large Angular or React project?

**Expected answer:**

Prefer organization around:

- Features.
- Business domains.
- Shared components.
- API/data layers.

Avoid one large generic folder containing everything.

---

### 16. How should authentication tokens be handled in frontend applications?

**Expected answer:**

The candidate should understand:
```

```
- XSS risks.
- Cookie/storage trade-offs.
- Token exposure.
- CSRF where applicable.

---

### 17. How would you handle API errors consistently?

**Expected answer:**

Centralize common error handling and convert technical errors into meaningful UI
states.

---

### 18. How do you optimize rendering of thousands of rows?

**Expected answer:**

Use virtualization where appropriate.

Avoid rendering thousands of unnecessary DOM elements.

---

### 19. What testing strategy would you use for a frontend application?

**Expected answer:**

Use a combination of:

- Component/unit tests.
- Integration tests.
- Focused end-to-end tests.

---

### 20. Describe a difficult frontend performance or state-management problem
you solved.

**Expected answer:**

Look for real debugging experience.

---
```

11. HTML / CSS / JavaScript / jQuery

Basic

1. What is semantic HTML and why is it important?

Expected answer:

Semantic elements describe the meaning of content.

Examples:

- `header`
- `nav`
- `main`
- `button`

Benefits include accessibility and maintainability.

---

2. Explain the CSS box model.

Expected answer:

The box model contains:

- Content.
- Padding.
- Border.
- Margin.

---

3. When would you use Flexbox and when would you use Grid?

Expected answer:

Flexbox is useful for primarily one-dimensional layouts.

Grid is useful for two-dimensional layouts.

---

4. What is event bubbling?

Expected answer:

````
An event can propagate from the target element upward through its parent
elements.

This allows techniques such as event delegation.

---

### 5. What is the difference between `var`, `let`, and `const`?

**Expected answer:**

`let` and `const` are block scoped.

`const` prevents reassignment.

`var` has older function-scoping behavior and should generally be avoided in
modern JavaScript.

---

### 6. What is the difference between `==` and `===`?

**Expected answer:**

`==` can perform type coercion.

`===` compares without coercion and is generally safer.

---

## Intermediate

### 7. What is a closure?

**Expected answer:**

A function can retain access to variables from the scope where it was created.

Closures are useful but can also unintentionally retain references.

---

### 8. Can you explain how the JavaScript event loop works?

**Expected answer:**

The candidate should understand:
````

```
- Call stack.
- Task queue.
- Microtasks.
- Promise callbacks.

Exact implementation details are less important than understanding asynchronous
ordering.

---

### 9. What is event delegation?

**Expected answer:**

Attach an event handler to a common parent instead of every child.

Useful for dynamic elements.

---

### 10. What is the difference between debounce and throttle?

**Expected answer:**

Debounce waits until activity stops.

Throttle limits how frequently an operation can run.

Example:

Search input → debounce.

Scroll event → often throttle.

---

### 11. What is CORS?

**Expected answer:**

CORS is a browser security mechanism controlling cross-origin requests based on
server response headers.

It is not an authentication system.

---
```

```
### 12. What is XSS?

**Expected answer:**

Cross-Site Scripting occurs when malicious script is executed in a user's
browser.

Reduce risk using:

- Output encoding.
- Safe DOM APIs.
- Sanitization where needed.
- Content Security Policy.

---

## Advanced

### 13. Why might a web page take a long time before the user sees useful
content?

**Expected answer:**

Possible causes:

- Large JavaScript bundles.
- Blocking scripts.
- Slow CSS.
- Large images.
- Slow API calls.

Use browser profiling tools to investigate.

---

### 14. How would you investigate a frontend memory leak?

**Expected answer:**

Use browser developer tools and check:

- Detached DOM nodes.
- Event listeners.
- Closures.
- Repeated component creation.

---
```

15. How does CSS specificity affect maintainability?

Expected answer:

Excessively specific selectors can make overrides difficult.

Avoid escalating specificity unnecessarily.

---

16. When is jQuery still useful?

Expected answer:

It may still be appropriate for maintaining legacy applications.

For new applications, modern browser APIs and frameworks often remove the need for it.

---

17. How would you optimize a page containing thousands of DOM elements?

Expected answer:

Consider:

- Virtualization.
- Reducing unnecessary rendering.
- Batching DOM operations.
- Profiling layout and paint.

---

18. Is client-side validation enough for security?

Expected answer:

No.

Server-side validation and authorization are authoritative.

Client validation mainly improves user experience.

---

19. What strategy would you use for browser compatibility?

Expected answer:

Define supported browsers and use:

- Modern standards.
- Transpilation.
- Polyfills where required.
- Testing for critical flows.

---

20. Tell me about a difficult JavaScript or browser issue you debugged.

Expected answer:

Look for practical debugging ability.

---

12. Unit Testing

Basic

1. What is a unit test?

Expected answer:

A unit test verifies a small piece of behavior in isolation.

It should usually be:

- Fast.
- Deterministic.
- Focused.

---

2. What is Arrange, Act, Assert?

Expected answer:

A common test structure:

1. Arrange data and dependencies.
2. Act by calling the behavior.
3. Assert the expected result.

```
### 3. What makes a good unit test?

**Expected answer:**

A good test should be:

- Readable.
- Deterministic.
- Focused.
- Meaningful.
- Able to fail when behavior is broken.

---

### 4. What is mocking?

**Expected answer:**

Mocking replaces or controls dependencies so the unit can be tested
independently.

Do not mock everything automatically.

---

### 5. What is the difference between a stub and a mock?

**Expected answer:**

A stub provides controlled data.

A mock can also verify interactions.

The exact terminology can vary.

---

### 6. Should you unit test every line of code?

**Expected answer:**

No.

Focus on meaningful behavior and important business logic.

---
```

```
## Intermediate

### 7. When would you use an integration test instead of a unit test?

**Expected answer:**

Use integration tests when correctness depends on real components working
together.

Examples:

- Database mapping.
- HTTP pipeline.
- Serialization.
- Authentication.
- External integration contracts.

---

### 8. What causes flaky tests?

**Expected answer:**

Common causes:

- Timing.
- Randomness.
- Shared state.
- External dependencies.
- Concurrency.
- Test order.

---

### 9. How do you test asynchronous code?

**Expected answer:**

Await the operation.

Avoid arbitrary `Thread.Sleep` or fixed delays.

Control dependencies where possible.

---

### 10. How do you test exceptions?
```

Expected answer:

Verify that the expected exception is thrown and assert meaningful information where necessary.

---

11. What is code coverage?

Expected answer:

It measures how much code was executed by tests.

High coverage does not guarantee good tests.

---

12. How would you test a service that depends on a repository?

Expected answer:

Unit test business logic by controlling the repository dependency.

Also use integration tests for the actual repository implementation.

---

13. What are test doubles?

Expected answer:

A general term covering:

- Mocks.
- Stubs.
- Fakes.
- Spies.
- Dummies.

Advanced

14. How would you test code that depends on the current date and time?

Expected answer:

Avoid directly depending on system time everywhere.

````
Use a time abstraction such as a provider so tests can control time.

Example concept:

```csharp id="q38zsa"
public class SubscriptionService
{
    private readonly TimeProvider _timeProvider;
}
````

Tests can use a controlled time.

---

15. How would you test retry logic?

Expected answer:

Test:

- Successful first attempt.
- Failure followed by success.
- Maximum retry reached.
- Correct error classification.

Avoid tests that actually wait several seconds.

---

16. How do you know whether your tests are actually good enough to catch bugs, beyond just looking at code coverage?

Expected answer:

Look at whether tests fail when meaningful behavior is changed.

Mutation testing is one technique that intentionally changes code to see whether tests detect the change.

---

17. What is over-mocking?

Expected answer:

Testing implementation details through many mock interaction assertions instead of testing meaningful behavior.
````

This can make tests fragile during refactoring.

---

18. How would you test a message consumer?

Expected answer:

Test:

- Valid messages.
- Invalid messages.
- Duplicate messages.
- Retry behavior.
- Failure handling.
- Idempotency.

Integration testing should also verify real serialization and infrastructure behavior.

---

19. What would you look for when reviewing tests in a pull request?

Expected answer:

Check:

- Important behavior.
- Edge cases.
- Failure paths.
- Meaningful assertions.
- Deterministic behavior.
- Regression protection.

---

20. What testing strategy would you recommend for a large application?

Expected answer:

Use a balanced approach:

- Many focused unit tests.
- Meaningful integration tests.
- Limited end-to-end tests for critical workflows.

Do not rely entirely on one type of test.

Additional Topics Strongly Recommended for a 6+ Year Developer

The original topics are good, but I recommend adding these sections.

1. System Design

Ask scenarios such as:

- Design a scalable notification system.
- Design an API handling millions of requests.
- Design a file processing system.
- Design a reliable background job system.
- How would you introduce caching?
- How would you handle failure and recovery?

This is one of the most important areas for a 6+ year developer.

---

2. Security

Cover:

- JWT.
- OAuth/OpenID Connect.
- Authentication vs authorization.
- OWASP basics.
- SQL injection.
- XSS.
- CSRF.
- Secrets.
- API security.
- RBAC and claims.

---

3. Design Patterns

Focus on practical usage:

- Strategy.
- Factory.
- Adapter.
- Decorator.
- Repository.

- CQRS.
- Mediator.

Most importantly ask:

> When would you not use this pattern?

---

4. Performance and Production Troubleshooting

Ask real scenarios:

> An API suddenly becomes slow in production. Where do you start?
>
> CPU is normal but requests are timing out. What would you investigate?
>
> Memory keeps increasing over several days. How would you debug it?

These questions are highly valuable for measuring real experience.

---

5. DevOps and Deployment

Cover:

- Docker.
- CI/CD.
- Environment configuration.
- Rollback.
- Health checks.
- Blue-green deployment.
- Kubernetes basics if relevant.

Recommended Interview Structure

For a 60–90 minute interview, do not ask all 240 questions.

A practical structure:

Area	Time
C#, OOP, SOLID	15 minutes
.NET Core and API design	15 minutes

Area	Time
SQL and LINQ	15 minutes
Microservices and Cloud	15 minutes
Testing and AI tools	10 minutes
Real-world scenario	10–20 minutes

Final Interview Advice

For a developer with 6+ years of experience, definitions alone are not enough.

The strongest questions are scenario-based.

Instead of asking:

> What is idempotency?

Ask:

> Your payment API receives the same request twice because the client timed out and retried. How would you make sure the customer is not charged twice?

Instead of asking:

> What is a deadlock?

Ask:

> Two database operations occasionally fail in production with a deadlock error. What would you investigate and how would you reduce the problem?

Instead of asking:

> What is Dependency Injection?

Ask:

> You have a service that directly creates its own database repository and email service. What problems could this create, and how would you improve the design?

This approach measures whether the candidate can **recognize the problem, reason about it, explain the solution, and discuss trade-offs**.

That is the most useful signal when interviewing a 6+ years .NET developer.